

UX/UI

Introdução a UX/UI

Objetivo:

Compreender os conceitos fundamentais de UX (User Experience) e UI (User Interface), identificando boas práticas de design e usabilidade para aplicações web e móveis.

1. O que é UX e UI?

UX (User Experience - Experiência do Usuário)

A experiência do usuário está relacionada à forma como um usuário interage com um sistema, site ou aplicativo. Uma boa UX torna a interação intuitiva, eficiente e agradável.

Principais aspectos:

- Usabilidade: facilidade de uso e acessibilidade.
- Arquitetura da informação: organização do conteúdo para melhor navegação.
- Acessibilidade: inclusão de diferentes tipos de usuários (ex.: pessoas com deficiência visual).
- Design centrado no usuário: desenvolver soluções baseadas nas necessidades reais dos usuários.

UI (User Interface - Interface do Usuário)

A interface do usuário se refere à parte visual e interativa do sistema. Ela define a aparência e o funcionamento dos elementos com os quais o usuário interage.

Principais elementos:

- Cores e tipografia: impacto visual e legibilidade.
- Botões e ícones: clareza e responsividade.

- Layout e espaçamento: harmonia visual e navegabilidade.
- Feedback visual: animações e transições para orientar o usuário.

Diferença entre UX e UI

Característica	UX (Experiência do Usuário)	UI (Interface do Usuário)
Objetivo	Melhorar a experiência geral do usuário	Criar interfaces visuais atraentes e funcionais
Envolve	Pesquisa de usuários, prototipagem, testes	Design visual, interatividade, responsividade
Impacto	Determina a satisfação e engajamento do usuário	Afeta a usabilidade e estética do sistema

2. Princípios de UX/UI

Heurísticas de Nielsen

1. **Visibilidade do status do sistema** – O usuário deve sempre saber o que está acontecendo.
2. **Correspondência entre sistema e mundo real** – Uso de termos familiares ao usuário.
3. **Controle e liberdade do usuário** – Facilidade para desfazer ou refazer ações.
4. **Consistência e padrões** – Manter elementos familiares em toda a interface.
5. **Prevenção de erros** – Criar avisos antes de ações irreversíveis.
6. **Reconhecimento em vez de recordação** – Evitar que o usuário tenha que memorizar informações.
7. **Flexibilidade e eficiência** – Criar atalhos e recursos para diferentes tipos de usuários.
8. **Estética e design minimalista** – Evitar poluição visual e excesso de informação.
9. **Ajuda ao usuário** – Fornecer documentação clara e acessível.

Acessibilidade e Inclusão

- **Cores acessíveis:** contraste adequado para deficientes visuais.
 - **Navegação alternativa:** uso de teclados e leitores de tela.
 - **Texto descritivo:** legibilidade e tamanhos ajustáveis.
-

3. Ferramentas para UX/UI

- **Figma** – Criação de protótipos interativos.
 - **Adobe XD** – Design de interfaces digitais.
 - **Sketch** – Design para macOS, com foco em UX/UI.
 - **Canva** – Criação de layouts simples e intuitivos.
-

4. Situação de Aprendizagem: Redesign de um Aplicativo

Atividade Prática (40 min)

Os alunos serão divididos em grupos e receberão a tarefa de analisar um aplicativo ou site popular (exemplo: uma loja virtual ou uma rede social). Eles devem identificar problemas de usabilidade e propor soluções de UX/UI.

Passos da Atividade:

1. Escolher um aplicativo ou site e analisar sua interface.
2. Listar problemas de UX/UI observados.
3. Propor soluções para melhorar a experiência do usuário.
4. Criar um protótipo de baixa fidelidade (desenho em papel ou ferramenta digital).

5. Apresentar soluções para a turma.

Objetivos da Atividade:

- Identificar falhas em interfaces reais.
- Propor soluções com base nas heurísticas de Nielsen.
- Estimular o pensamento crítico e colaborativo.

Entrega

G1 - Bosso, Enzo, Gabriel, Beltramo - moviecom

G2 - Ana, Alice - Shein

G3 - Evelyn. Ana S , Emily, Gabriela - Blinkee.com

G4 - Vinicius, Thayssa, Lucas, Barbara - Biblioteca Americana

G5 - Duda, João, Larissa , Carol, Davi - Site do SESI - Secretaria digital

G6 - LArisa N, Lorenzo, Rian, Murilo -

shopping queen annie

G7 - LAura, Pedro L - Secretaria Digital SENAI

G8 - Guilherme, Eduardo, Filipe, Pedro C - GCC

5. Conclusão e Reflexão

- UX/UI impactam diretamente a experiência do usuário e a aceitação de um produto.

- Pequenas mudanças podem melhorar drasticamente a usabilidade de um site ou aplicativo.
- Pensar no usuário é essencial para o sucesso de qualquer projeto digital.

Dica Final: A melhor forma de aprender UX/UI é praticando! Analise interfaces no seu dia a dia e tente identificar melhorias.

Recursos Complementares:

- "Don't Make Me Think" – Steve Krug (Livro sobre usabilidade)
- Nielsen Norman Group – <https://www.nngroup.com>
- Material Design (Google) – <https://material.io>
- Apple Human Interface Guidelines – <https://developer.apple.com/design/human-interface-guidelines/>

Link Para Reuniões

Frameworks

Desenvolvimento com Frameworks Front-End

O que são frameworks?

Frameworks são conjuntos de bibliotecas e padrões que fornecem uma estrutura para o desenvolvimento de aplicações. Eles ajudam a organizar e otimizar o código, facilitando a manutenção e escalabilidade dos projetos.

Diferença entre bibliotecas e frameworks

Característica	Biblioteca (Ex: jQuery)	Framework (Ex: Angular, React, Vue)
Controle	O desenvolvedor controla o fluxo do código	O framework dita o fluxo do código
Estrutura	Não impõe estrutura	Define padrões e estrutura fixa
Flexibilidade	Mais flexível, usado conforme necessário	Menos flexível, pois segue regras
Dependência	Pode ser usada pontualmente	Estrutura completa para o projeto

Principais vantagens do uso de frameworks

Reutilização de código – Componentes reaproveitáveis reduzem esforço e tempo.

Facilidade de manutenção – Aplicações bem estruturadas com separação de responsabilidades.

Comunidade ativa – Suporte contínuo, novas funcionalidades e atualizações frequentes.

Performance – Frameworks modernos utilizam técnicas de otimização, como Virtual DOM (React) e Two-Way Binding (Vue, Angular).

Integração com ferramentas – Fácil integração com APIs, banco de dados e bibliotecas externas.

Frameworks de Front-End

O **Front-End** é a parte da aplicação com a qual o usuário interage diretamente. Ele é responsável por exibir a interface do usuário (UI), lidar com interações e tornar a experiência fluida e intuitiva.

Principais tecnologias usadas no Front-End:

- **HTML (HyperText Markup Language)** – Estrutura da página web.
- **CSS (Cascading Style Sheets)** – Estilização e design visual.
- **JavaScript** – Funcionalidade e interatividade.

1. React

- Biblioteca para construção de interfaces de usuário (UI) interativas e dinâmicas.
- Mantido pelo Facebook (Meta), amplamente utilizado no mercado.
- Baseado em componentes reutilizáveis, facilitando a manutenção e escalabilidade.
- Ideal para Single Page Applications (SPA) e aplicações web modernas.

2. Angular

- Framework completo e robusto mantido pelo Google.
- Utilizado para desenvolvimento de aplicações web escaláveis e complexas.
- Oferece recursos integrados como roteamento, injeção de dependências e manipulação avançada do DOM.
- Segue o padrão MVC (Model-View-Controller), promovendo organização e modularidade.

3. Vue.js

- Framework progressivo para construção de interfaces de usuário.
- Conhecido por sua curva de aprendizado suave e fácil integração com projetos existentes.
- Pode ser utilizado tanto para aplicações pequenas quanto para projetos complexos e escaláveis.
- Oferece reatividade eficiente e uma sintaxe intuitiva.

Frameworks de Back-End

O **Back-End** é a parte da aplicação que fica no servidor, responsável pelo processamento de dados, autenticação, segurança e regras de negócio. Ele recebe requisições do Front-End, processa as informações e devolve os resultados.

Principais tecnologias usadas no Back-End:

- **Node.js** – Ambiente para rodar JavaScript no servidor.
- **Linguagens como Python, Java, PHP, Ruby** – Usadas para criar lógica de negócios.
- **Bancos de dados (SQL e NoSQL)** – Armazenam e gerenciam dados.

1. Express.js

- Framework minimalista e flexível para Node.js.
- Amplamente utilizado para criação de servidores web e APIs RESTful.
- Permite middleware personalizados, facilitando a manipulação de requisições e respostas.

2. Meteor

- Plataforma completa para desenvolvimento web e mobile.
- Facilita a criação de aplicações em tempo real com sincronização automática de dados.
- Integra front-end e back-end, reduzindo a complexidade no desenvolvimento.

3. Nest.js

- Framework moderno para back-end em TypeScript, baseado no Node.js.
- Segue a arquitetura modular e suporta Microservices.
- Inspirado em Angular, com injeção de dependências e estrutura escalável.

Gerenciamento de Estado

O **estado** de uma aplicação representa os dados armazenados no momento. O gerenciamento de estado é crucial para manter a consistência das informações exibidas ao usuário.

1. Redux

- Biblioteca para gerenciamento de estado previsível em aplicações React.
- Utiliza um único store para armazenar o estado global da aplicação.
- Baseado no conceito de ações e reducers, garantindo imutabilidade e previsibilidade.

2. MobX

- Alternativa ao Redux para gerenciamento de estado em JavaScript.
- Adota uma abordagem reativa e baseada em observáveis.
- Menos verboso e mais flexível que o Redux, permitindo atualizações automáticas do estado.

Frameworks de Renderização do Lado do Servidor (SSR)

No SSR, as páginas são geradas no servidor antes de serem enviadas ao navegador. Isso melhora o desempenho e o SEO (otimização para mecanismos de busca).

1. Next.js

- Framework para renderização do lado do servidor (SSR) em aplicativos React.
- Melhora o desempenho, tempo de carregamento e SEO das páginas.
- Suporte a geração estática (SSG) e rendering híbrido para otimização de performance.

2. Nuxt.js

- Framework para renderização do lado do servidor (SSR) em aplicações Vue.js.

- Estrutura semelhante ao Next.js, oferecendo SEO aprimorado e melhor performance.
- Facilita a organização do projeto e suporte a geração de páginas estáticas.

Obs: SPA (Single Page Application)

As SPAs carregam apenas um único arquivo HTML e atualizam a interface dinamicamente sem recarregar a página. Isso torna a navegação mais fluida, mas pode prejudicar o SEO.

Frameworks como **React**, **Angular** e **Vue** são amplamente usados para criar SPAs.

Comparação entre React, Angular e Vue

Característica	React (Meta)	Angular (Google)	Vue (Comunidade)
Linguagem	JavaScript/JSX/ TypeScript	TypeScript	JavaScript/Typescript
Paradigma	Baseado em componentes	Estrutura MVC	Baseado em componentes
Curva de aprendizado	Média	Alta (arquitetura complexa)	Baixa (sintaxe simples)
Virtual DOM	Sim	Não	Sim
Data Binding	Unidirecional	Bidirecional	Bidirecional
Popularidade	Alta	Média	Crescente
Casos de uso	Aplicações dinâmicas e SPA	Aplicações empresariais grandes	Aplicações ágeis e leves
Empresas que usam	Facebook, Instagram, Airbnb	Google, Microsoft, IBM	Alibaba, Xiaomi, GitLab

Ferramentas auxiliares

Vite – Build ultrarrápido para React e Vue

Webpack – Empacotador de módulos usado no Angular

Babel – Transpiler para código JavaScript moderno

Exemplos de uso no mercado

- **React:** Aplicações modernas, redes sociais e dashboards dinâmicos (Ex: Facebook, Instagram, Netflix).
 - **Angular:** Aplicações corporativas e sistemas escaláveis (Ex: Google Drive, Gmail, Microsoft Office Online).
 - **Vue:** Aplicações ágeis e interativas (Ex: Alibaba, Xiaomi, GitLab).
-

Conclusão

Os frameworks **React**, **Angular** e **Vue** são amplamente utilizados no mercado, cada um com suas vantagens. O uso correto depende do tipo de aplicação e dos requisitos do projeto.

Introdução Angular

Objetivos:

Compreender os tipos de Frameworks em JavaScript - Angular

Conteúdo:

1. Principais funcionalidades
2. Instalação e Aplicação

Metodologia e Estratégia:

1. Aula expositiva dialogada com apoio de tutorial;
2. Exercícios de aplicação.

Framework Angular

Angular é um framework de desenvolvimento de aplicações web de código aberto criado pelo Google. Ele é uma das escolhas mais populares para o desenvolvimento de aplicações web robustas, escaláveis e dinâmicas. A seguir, destacamos suas principais funcionalidades:

Principais Funcionalidades

- Arquitetura baseada em componentes: Divide a interface do usuário em componentes reutilizáveis e independentes.
- TypeScript: Linguagem que adiciona tipagem estática e recursos avançados ao JavaScript.
https://docs.google.com/document/d/1M0S869YnbD3608UMX5g2tLp3XdfttqiT5tne_6oXf5Q/edit?tab=t.a5fn7xarbncl
- Two-Way Data Binding: Permite a sincronização automática entre a camada de modelo e a interface do usuário.
- Diretivas: Instruções no HTML que estendem o comportamento da marcação.

- Injeção de dependência: Facilita a gestão de dependências e torna o código modular.
- Routing: Cria aplicações de múltiplas páginas com navegação eficiente.
- Pipes: Transformam dados exibidos no modelo.
- Comunicação HTTP: Usa o HttpClient para acessar APIs externas.
- Testabilidade: Ferramentas para escrita de testes unitários e de integração.
- PWAs e SPAs: Suporte para Progressive Web Apps e Single Page Applications.

Instalação e Configuração do Angular

Requisitos:

- Node.js instalado (verifique com `node -v` e `npm -v` no terminal).

Instalar o Angular CLI

```
npm install -g @angular/cli
```

Criar um Novo Projeto Angular

```
ng new nome-do-projeto
```

Caso haja bloqueios de execução de script, tente:

```
npx ng new nome-do-projeto
```

Se persistir, altere a política de execução no PowerShell:

```
Set-ExecutionPolicy Unrestricted
```

Navegar para o Diretório do Projeto

```
cd nome-do-projeto
```

Iniciar o Servidor de Desenvolvimento

```
ng serve
```

Ou

```
npm start
```

Acesse: <http://localhost:4200>

Instalar Dependências Adicionais

Exemplo de Instalação do bootstrap

```
npm install bootstrap
```

Criando Header, Footer e Páginas de Navegação

Criar Componentes

```
ng generate component components/header
```

```
ng generate component components/footer
```

```
ng generate component pages/home
```

```
ng generate component pages/products
```

```
ng generate component pages/contact
```

Configurar as Rotas

Edite *app-routing.module.ts*:

https://docs.google.com/document/d/1dfvSQd_HZ2qomL_Vq6toz5K9WHFiSDzu49q-yWUpzzM/edit?tab=t.7jqy8jkixtfv

Criar o Header e o Footer

- ☐ Edite header.component.html:
- ☐ Estilizando o Header (header.component.css)
- ☐ Footer (footer.component.html)
- ☐ Estilizando o Footer (footer.component.css)

Configurar o app.component.html

Agora, inclua o Header e o Footer no app.component.html para ficarem fixos em todas as páginas:

```
<app-header></app-header>  
<router-outlet></router-outlet>  
<app-footer></app-footer>
```

Criar o Conteúdo das Páginas

- ☐ Home (home.component.html)
- ☐ Estilizando (home.component.css)
- ☐ Products (products.component.html)
- ☐ Estilizando (products.component.css)
- ☐ Contato (contact.component.html)
- ☐ Estilizando (products.component.css)

Data Binding no Angular

Conceitos

O conceito de data-binding (tradução: ligação de dados) está presente em várias linguagens, é a técnica de unir várias fontes de dados e deixá-las sincronizadas sem precisar ficar alterando os elementos da DOM manualmente.

Em resumo, basta realizar apenas uma alteração de valor e todos os lugares que fazem uso dessa variável irá refletir seu valor, no template e componente.

O Data Binding é o mecanismo que permite a comunicação entre a lógica da aplicação (TypeScript) e a interface do usuário (HTML). No Angular, existem quatro tipos principais de Data Binding:

1. Interpolation (Interpolação) → Passa valores do TypeScript para o HTML.
2. Property Binding → Define propriedades de elementos HTML dinamicamente.
3. Event Binding → Responde a eventos do usuário, como cliques e teclas pressionadas.
4. Two-Way Data Binding → Mantém a sincronização entre o modelo e a interface.

Interpolation - Interpolação - {{ }}

Muitas vezes, queremos que alguns conteúdos do nosso template.html tenha informações dinâmicas ou que venham de algum outro lugar da aplicação.

Para isso, podemos ter uma variável em nosso component.ts com o conteúdo desejado, e exibir em tela, através do nosso template.html.

A maneira mais simples, é passar esses dados através do que chamamos de interpolação, no Angular, podemos fazer uso de: {{ nome-variavel }}.

Primeiramente, podemos definir algumas variáveis em nosso component.ts, note que a tipagem é opcional.

```
export class AppComponent {  
  nome: string = "Maria";
```

```
}
```

Após a declaração, podemos recuperar essas informações e exibi-las em tela através da sintaxe de interpolação do Angular `{{ nome-variavel }}`.

Abaixo, segue um exemplo de como isso pode ser feito em nosso [template.html](#).

```
<h1>Olá, {{ nome }}!</h1>
```

Após a compilação da aplicação, esses serão os dados exibidos em tela. Note que além de ser possível exibir o conteúdo das variáveis, também podemos executar operações e condições.

Property Binding ([])

É uma forma de atribuirmos valores aos atributos/propriedades de um componente. Temos duas formas de fazer isso, através de interpolação e pelo property binding em si.

```
export class AppComponent {  
  imgUrl: string = "https://angular.io/assets/images/logos/angular/angular.svg";  
}
```

Para exibir em tela, criamos a tag de imagem em nosso **template.html**, onde:

```
<img [src]="imgUrl" width="150">
```

- ◆ Explicação: A imagem será carregada a partir da URL definida no TypeScript.

Class e Style Binding

Assim como os atributos, também conseguimos realizar atribuições de classes após realizar alguma ação. No exemplo abaixo, temos a definição de classes por interpolação e por style binding.

Nesse caso, trabalharemos apenas no template.html, mas pode ser que sua aplicação controle os valores através do component.ts. Primeiramente, criamos um <select> e suas <option>, os valores correspondem as classes do bootstrap.

A variável que está sendo utilizada, é a #classe, variável de template que já explicamos no início desse artigo, e tem como objetivo pegar o valor atual do <select> utilizando classe.value.

- Caso 1 (utilizando interpolação): Temos uma <div>, onde dentro do seu atributo class, está recebendo uma classe, baseada no <select>, onde está sendo passada por interpolação <div class="{{ classe.value }}"></div>.
- Caso 2 (utilizando class binding): Nesse caso, utilizamos o class binding para atribuição de classe, utilizando como exemplo [class.alert-success]. Onde o class é o binding e o alert-success é o nome da classe que queremos atribuir. Além disso, ainda estamos fazendo uma operação, onde só irá ser atribuída a classe, caso o valor do <select> corresponda a ela.
- Caso 3 (utilizando style binding com operador ternário): Também é possível trabalhar diretamente com um estilo do css, utilizando style binding. No exemplo, utilizamos o [style.display] com um ternário, onde eu só quero que exiba essa <div> caso seja selecionado a opção de Erro (alert-danger).

```
<article>
```

```
  <h3>Class e Style Binding</h3>
```

```
  <p>Selecione uma classe</p>
```

```
  <select #classe (change)="0">
```

```
    <optgroup label="Alertas">
```

```
      <option value="alert-success">Sucesso</option>
```

```
      <option value="alert-info">Info</option>
```

```
      <option value="alert-warning">Alerta</option>
```

```
      <option value="alert-danger">Erro</option>
```

```
    <<optgroup>
```

```
  </select>
```

```
<br><br>
```

```
<!-- Interpolação -->
```

```

<div class="alert {{ classe.value }}" role="alert">
  Texto colorido conforme o valor do combobox
</div>

<!-- Property -->
<div class="alert" [class.alert-success]="classe.value = 'alert-success'" role="alert">
  A simple success alert-check it out!
</div>

<div class="alert" (class.alert-info)="classe.value = 'alert-info'" role="alert">
  A simple info alert-check it out!
</div>

<div class="alert" [class.alert-warning]="classe.value = 'alert-warning'" role="alert">
  A simple warning alert-check it out!
</div>

<div class="alert" [class.alert-danger]="classe.value = 'alert-danger'" role="alert">
  A simple danger alert-check it out!
</div>

<!-- Property com ternário -->
<div class="alert alert-danger" [style.display]="classe.value = 'alert-danger'? 'block':
'none'" role="alert">
  Esse texto somente aparece em caso de erro.
</div>

</article>

```

Event Binding (())

É uma forma de associar eventos, que devem ser executados através de uma determinada ação do usuário e comunicar com uma classe/função presente no component.ts.

Para os eventos, utilizamos a sintaxe (evento)="funcaoDisparada()", onde (evento) é o tipo do evento da DOM a ser disparado, lista completa aqui, e a função é a que criaremos em nosso **component.ts**.

```
export class AppComponent {  
  contador: number = 0;  
  isMouseOver: boolean = false  
  
  incrementar() {  
    this.contador++;  
  }  
  
  onKeyUp(event: KeyboardEvent) {  
    this.valorAtual += event.key;  
  }  
  
  salvarvalor (event: any) {  
    this.valorSalvo = event;  
  }  
  
  onMouseOverOut() {  
    this.isMouseOver = !this.isMouseOver  
  }  
}
```

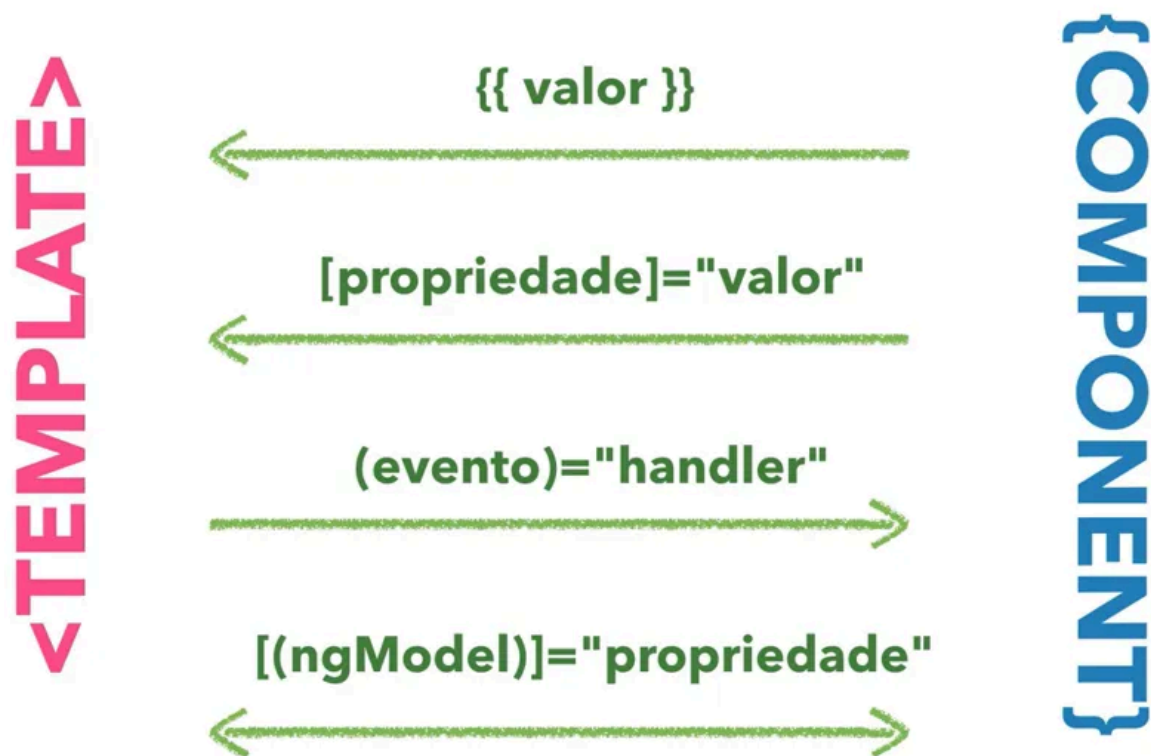
template.html

```
<section>  
  <article>  
    <h3>Event Binding</h3>  
    <div>  
      <button class="btn btn-primary" (click)="botaoClicado()">Me  
clique!</button>
```

```
<br><br>
<input type="text"
      (keyup)="onKeyUp($event)"
      (keyup.enter)="salvarValor (campoInput.value)"
      (blur)="salvarValor (campoInput.value)"
      #campoInput
>
<p>Valor atual: {{ valorAtual }}</p>
<p>Valor salvo: {{ valorSalvo }}</p>
<br><br>
<span
      (mouseover)="onMouseOverOut()"
      (mouseout)="onMouseOverOut()"
      [class.highlight]="isMouseOver"
>
Passe o mouse sobre mim
</span>
<br><br>
<button (click)="incrementar()">Clique aqui</button>
<p>Contagem: {{ contador }}</p>

</div>
</article>
</section>
```

Two-Way Data Binding – [(ngModel)]



Até o presente momento, todos os exemplos de binding que aprendemos, são one-way data binding (vinculação de dados unidirecional), ou seja, o dado quando é atualizado, só reflete em um lado da aplicação. Como no exemplo abaixo.

O Two-Way Data Binding mantém sincronizados os dados entre a interface e o TypeScript. Quando utilizamos o two-way data binding (vinculação de dados bidirecional), as atualizações desses dados são refletidas em todos os lugares da aplicação, não importa onde esteja sendo utilizado.

Para isso, no caso de formulários, utilizamos uma diretiva do módulo FormsModule (é necessário o módulo estar importado) chamada ngModel.

Note que para fazer uso do two-way data binding, precisamos realizar o binding de propriedade e de evento ao mesmo tempo, para isso, utilizamos o atalho: [(ngModel)].

No exemplo abaixo, qualquer alteração que eu faça no elemento <input> ele está refletindo no restante da aplicação, atualizando também a interpolação realizada no template.html.

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent {  
  nome: string = "João";  
}
```

template.html

```
<input type="text" [(ngModel)]="nome">  
<p>Olá, {{ nome }}!</p>
```

- ♦ Explicação: Conforme o usuário digita, a variável nome é atualizada automaticamente.

Observação: Para usar ngModel, é necessário importar o FormsModule no **app.module.ts**:

```
import { FormsModule } from '@angular/forms';
```

```
@NgModule({  
  imports: [BrowserModule, FormsModule],  
  declarations: [AppComponent],  
  bootstrap: [AppComponent]  
})
```

Exercício Prático: Criar uma aplicação de edição de formulário

Objetivo: Criar uma aplicação onde o usuário pode digitar um Formulário (Nome, Email, Telefone, Gênero, Idade, Profissão) e ver uma div com os dados sendo atualizado em tempo real.

- ☐ Criar o Projeto
- ☐ Criar o Componente
- ☐ Implementar a Lógica no Componente
- ☐ Criar o Template HTML
- ☐ Importar FormsModule
- ☐ Exibir o Componente

https://docs.google.com/document/d/1dfvSQd_HZ2qomL_Vq6toz5K9WHFiSDzu49q-yWUpzzM/edit?tab=t.bj6l7rxgrwu7

Sugestões de Melhorias

Validação de Campos (Angular Forms ou validações simples)

Verifique se os campos estão preenchidos.

Exiba mensagens como: "O campo nome é obrigatório!"

Botão "Limpar"

Um botão para resetar todos os campos do formulário.

Projeto Ordem de Pedidos

Vamos criar um projeto Angular 16 simples para demonstrar data binding (interpolação, property binding, event binding e two-way binding) no contexto de um cadastro básico de Clientes, Produtos e Pedidos. O objetivo é ver as atualizações em tempo real.

MVC com Angular

Projeto Angular 16: Sistema de Pedidos

Objetivo

Criar uma aplicação Angular para:

- Cadastrar **clientes**
 - Cadastrar **produtos**
 - Registrar **pedidos** vinculando clientes e produtos
 - Exibir os pedidos cadastrados com cálculo automático do total com desconto
-

1. Criação do Projeto

```
ng new pedidos-app --routing --style=scss
cd pedidos-app
ng generate component components/cliente-form
ng generate component components/produto-form
ng generate component components/pedido-form
ng generate component components/pedido-list
ng generate service services/dados
mkdir src/app/models
criar os arquivos para o model
cliente.model.ts / produto.model.ts / pedido.model.ts
```

2. Criação dos Modelos (POO)

[cliente.model.ts](#)

```
export class Cliente{
  id: number;
  nome: string;
```

```
constructor(id:number, nome: string){  
  this.id = id;  
  this.nome = nome;  
}  
}
```

Aplicação Completa Angular

Plataforma de RH (Frontend em Angular)

Objetivo:

Desenvolver uma plataforma web voltada para recrutamento e seleção, permitindo que usuários (candidatos) cadastrem seus dados e currículos, empresas registrem suas vagas, e que os candidatos se candidatem a essas vagas. O projeto será focado no frontend com Angular, utilizando o json-server como backend fake para simular a persistência dos dados.

Público-Alvo:

- Pessoas buscando oportunidades de emprego (usuários).
- Empresas ofertando vagas de trabalho.

Tecnologias:

- **Frontend:** Angular 17+
- **Backend Fake:** json-server
- **Estilização:** Angular Material (opcional, para UI amigável)
- **Comunicação HTTP:** HttpClientModule

Escopo do Projeto

Funcionalidades para Candidatos (Usuários):

- Cadastro/Login de usuário (simples, sem autenticação real)
- Edição de perfil e currículo (nome, email, habilidades, experiência, formação, etc.)
- Listagem de vagas disponíveis
- Candidatura às vagas

Funcionalidades para Empresas:

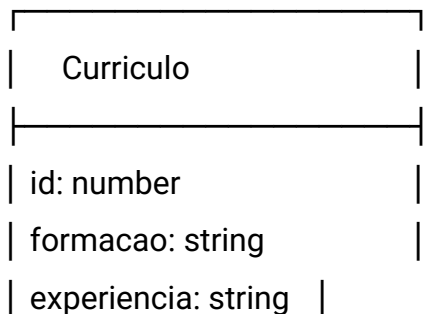
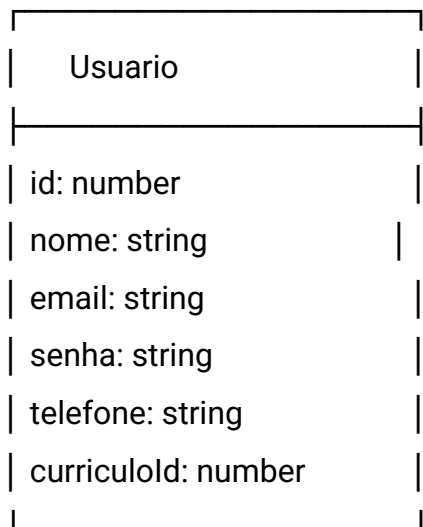
- Cadastro/Login de empresa (simples)
- Cadastro e edição de vagas (título, descrição, requisitos, local, tipo de contratação)

- Visualização de candidatos às vagas (simulado via json-server)

Funcionalidades Gerais:

- Roteamento entre telas (Angular Router)
- Navegação com menus contextuais
- Consumo de dados via json-server
- Validações de formulários (Template ou Reactive Forms)
- Exibição de feedback (toasts/alerts) ao salvar ou enviar dados

Diagrama de Classes (Modelos de Dados)



habilidades: string
linkedin: string

Empresa
id: number
nome: string
cnpj: string
email: string
setor: string



Vaga
id: number
titulo: string
descricao: string
requisitos: string
local: string
tipo: string
empresald: number

Candidatura

id: number
usuarioid: number
vagald: number
dataCandidatura: Date

Criando o BackEnd Fake

Antes de começar, certifique-se de ter instalado:

Node.js

Configurando um servidor local para hospedar os arquivos.

Para Começar Vamos Criar um Servidor para hospedagem de Arquivo usando o Json-Server.

https://docs.google.com/document/d/1M0S869YnbD3608UMX5g2tLp3XdfttqjT5tne_6oXf5Q/edit?tab=t.7oykml4on4bl

db.json para o json-server (com dados mockados)

Crie um arquivo db.json na raiz do projeto

```
{
  "usuarios": [
    {
      "id": 1,
      "nome": "João Silva",
```

```
"email": "joao@email.com",
"senha": "123456",
"telefone": "11999999999",
"curriculoid": 1
}
],
"curriculos": [
{
  "id": 1,
  "formacao": "Engenharia de Software - USP",
  "experiencia": "3 anos como desenvolvedor Fullstack",
  "habilidades": "Angular, Node.js, MySQL",
  "linkedin": "https://linkedin.com/in/joaosilva"
}
],
"empresas": [
{
  "id": 1,
  "nome": "Tech Solutions",
  "cnpj": "12.345.678/0001-99",
  "email": "contato@techsolutions.com",
  "setor": "Tecnologia"
}
],
"vagas": [
{
  "id": 1,
  "titulo": "Desenvolvedor Angular",
  "descricao": "Atuar no desenvolvimento frontend com Angular.",
  "requisitos": "Experiência com Angular 13+, RxJS e TypeScript.",
```

```
    "local": "Remoto",
    "tipo": "CLT",
    "empresald": 1
  },
],
"candidaturas": [
  {
    "id": 1,
    "usuariold": 1,
    "vagald": 1,
    "dataCandidatura": "2025-04-20"
  }
]
}
```

Passo a passo para configurar o json-server

Instalar o json-server globalmente (caso ainda não tenha):

```
npm install -g json-server
```

Rodar o servidor:

```
json-server --watch db.json --port 3000
```

O servidor estará disponível em: <http://localhost:3000>

Endpoints disponíveis:

- GET /usuarios
- GET /curriculos
- GET /empresas
- GET /vagas

- GET /candidaturas

Script de teste do json-server (test-server.js)

Você pode criar um script de teste usando axios para validar se tudo está rodando bem:

Instale o axios:

```
npm install axios
```

Crie um arquivo test-server.js com o conteúdo abaixo:

```
const axios = require("axios");

const API = "http://localhost:3000";

async function testAPI() {
  try {
    const usuarios = await axios.get(`${API}/usuarios`);
    console.log("Usuários:", usuarios.data);

    const empresas = await axios.get(`${API}/empresas`);
    console.log("Empresas:", empresas.data);

    const vagas = await axios.get(`${API}/vagas`);
    console.log("Vagas:", vagas.data);

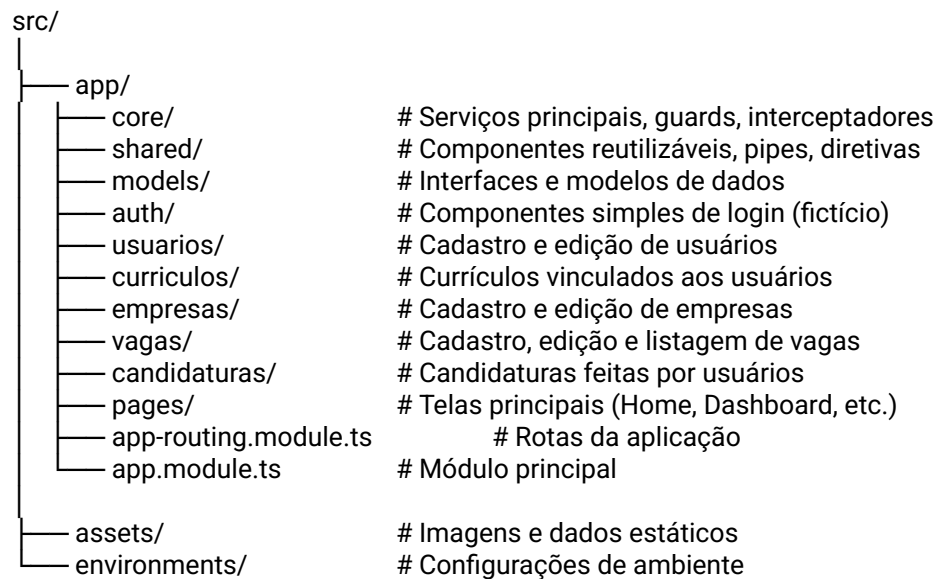
    const candidaturas = await axios.get(`${API}/candidaturas`);
    console.log("Candidaturas:", candidaturas.data);
  } catch (error) {
    console.error("Erro ao acessar API:", error.message);
  }
}
```

```
testAPI();
```

Execute com:

```
node test-server.js
```

Estrutura Geral do Projeto Angular



Etapas de Desenvolvimento

Etapa 1 - Criação do Projeto

```
ng new rh-platform --routing --style=scss
cd rh-platform
npm install
```

Instalar Angular Material (opcional para estilização):

ng add @angular/material

Etapa 2 - Gerar Estrutura de Módulos e Componentes

Etapa 3 - Criar Interfaces de Modelos (em /models)

Exemplo de usuario.model.ts:

```
export interface Usuario {  
  id: number;  
  nome: string;  
  email: string;  
  senha: string;  
  telefone: string;  
  curriculoid: number;  
}
```

Etapa 4 - Criar Serviços HTTP (em core/services)

Todos os serviços usarão HttpClient para consumir o json-server:

```
// exemplo base para todos  
constructor(private http: HttpClient) {}  
  
getAll(): Observable<Usuario[]> {
```

```
return this.http.get<Usuario[]>(`${this.api}/usuarios`);  
}
```

Etapa 5 - Configurar as Rotas (app-routing.module.ts)

Etapa 6 - Criar Formulários (Reactive Forms)

Exemplo de uso no usuario-form.component.ts:

```
this.form = this.fb.group({  
  nome: ['', Validators.required],  
  email: ['', [Validators.required, Validators.email]],  
  senha: ['', Validators.required],  
  telefone: [''],  
});
```

Etapa 7 - Conectar os serviços aos componentes

- ngOnInit() → chama o serviço para buscar dados
 - onSubmit() → salva ou edita
 - delete() → remove item
-

Etapa 8 - Integração entre Usuário ↔ Currículo, Empresa ↔ Vaga, Usuário ↔ Candidatura

- Ao cadastrar um usuário, redirecionar para preencher currículo
 - Ao cadastrar vaga, vincular com empresa
 - Botão “Candidatar-se” nas vagas para criar uma entrada em /candidaturas
-

Etapa 9 - Estilização e Componentes Visuais (Angular Material)

- Navbar com links principais
 - SnackBar para feedbacks (salvo, erro, etc.)
 - Card para listagem de vagas
 - Botões de ação (mat-button, mat-icon-button)
-

Etapa 10 - Testes básicos (manual) e verificação da integração com o

json-server

SA - Somativa - Portal RH

Situação de Aprendizagem: Plataforma de RH (Frontend Angular) - Módulo de Currículos

1. Visão Geral

Nesta situação de aprendizagem, você desenvolverá as funcionalidades relacionadas à gestão de currículos dentro de uma Plataforma de RH, utilizando Angular para o frontend e json-server para simular o backend. O objetivo é permitir que os usuários (candidatos) criem, editem e visualizem seus currículos, e que as empresas, de forma simulada, possam "visualizar" esses currículos vinculados às candidaturas. Este módulo de currículos é uma extensão natural do projeto principal, replicando a lógica de listagem e gerenciamento de dados já aplicada às vagas.

2. Objetivos de Aprendizagem

Ao final desta situação de aprendizagem, você será capaz de:

- Modelagem de Dados: Compreender e implementar as interfaces de dados para Currículo
- Serviços HTTP: Criar e utilizar serviços Angular (HttpClientModule) para interagir com o json-server, realizando operações CRUD (Create, Read, Update, Delete) em recursos de currículos.
- Formulários: Desenvolver formulários reativos para o cadastro e edição de currículos, aplicando validações e exibindo feedback ao usuário.
- Roteamento: Configurar e gerenciar rotas no Angular para navegação entre o cadastro de currículo e a listagem.
- Componentização: Organizar o código em componentes reutilizáveis para o formulário e a listagem de currículos.
- Reaproveitamento de Conhecimento: Aplicar os padrões e conhecimentos adquiridos no desenvolvimento das funcionalidades de vagas e empresas para o módulo de currículos.
- Experiência do Usuário (UI/UX): Implementar uma interface intuitiva para a gestão de currículos, utilizando (opcionalmente) Angular Material para uma melhor experiência.

3. Cenário

Imagine que você está desenvolvendo o módulo de currículos para a Plataforma de RH. Os usuários recém-cadastrados precisam ter um local para registrar suas informações

profissionais detalhadas (formação, experiência, habilidades), e essas informações precisam ser persistidas e visualizadas.

4. Escopo do Projeto - Módulo de Currículos

Funcionalidades para Candidatos (Usuários):

- Criação de Currículo: Um formulário para o usuário preencher seus dados de currículo (formação, experiência, habilidades, LinkedIn), vinculando-o ao `usuariold` do usuário logado (simulado).
- Edição de Currículo: Capacidade de editar as informações do currículo já existente.
- Visualização de Currículo: Uma tela onde o usuário pode ver seu próprio currículo.

Funcionalidades para ADMs(simuladas via json-server):

- Listagem de Currículos Vinculados: - Atualização e Exclusão de Curriculum Cadastrados
-

5. Pré-requisitos e Setup

Para iniciar, certifique-se de que o ambiente de desenvolvimento para o projeto principal já está configurado:

- Node.js instalado.
 - Angular CLI instalado globalmente (`npm install -g @angular/cli`).
 - json-server instalado globalmente (`npm install -g json-server`).
 - Projeto Angular (rh-platform) criado e com as dependências instaladas (`npm install`).
 - Arquivo `db.json` configurado na raiz do projeto com os dados mockados fornecidos.
 - Servidor json-server rodando: `json-server --watch db.json --port 30XX`
-

6. Passo a Passo do Desenvolvimento (Módulo de Currículos)

Siga estas etapas para implementar as funcionalidades de currículos, aplicando os mesmos princípios usados para o gerenciamento de vagas:

Etapa 1: Preparação da Estrutura

1. Gerar Model e Componentes:

- Crie um novo model para currículos:
- Dentro de app, crie os componentes:
 - curriculo-form (para cadastro e edição):
 - curriculo-list (listagem simulada para empresas):
 - curriculo-detail (opcional, para exibir detalhes de um currículo específico se a complexidade aumentar)

Etapa 2: Criar Serviço HTTP para Currículos

1. Gerar Serviço:

- Crie um serviço em `src/app/core/services/curriculo.service.ts`

2. Implementar Métodos CRUD:

- No `curriculo.service.ts`, implemente os métodos para interagir com o json-server (`/currículos`):
 - `getCurriculoByUsuarioId(usuarioId: number): Observable<Curriculo[]>` (json-server permite filtros)
 - `postCurriculo(curriculo: Curriculo): Observable<Curriculo>`
 - `putCurriculo(curriculo: Curriculo): Observable<Curriculo>`
 - `deleteCurriculo(id: number): Observable<void>` (se aplicável, para o usuário remover o próprio currículo)

Etapa 3: Desenvolver Componentes de Formulário e Listagem

1. CurriculoFormComponent:

- Implemente a lógica de salvar (criar ou atualizar) o currículo, chamando o `CurriculoService`.
- Exiba feedback ao usuário (usando `MatSnackBar` ou alerts simples).

2. CurrículoListComponent:

- Este componente terá dois propósitos:
 - Visualização do próprio currículo pelo usuário: Buscar o currículo do
 - Listagem simulada de currículos para empresas: Em uma rota separada, simule a "visualização de candidatos" a uma vaga

3. CurrículoDetailComponent(Opcional):

- Componente para exibir melhores detalhes do currículo, com mais informações, sobre o currículo do candidato

Etapa 4: Configurar Rotas

1. app-routing.module.ts :

- Adicione rotas para:
 - /curriculos/novo (para cadastrar um novo currículo)
 - /curriculos/editar/:id (para editar um currículo existente)
 - /meu-curriculo (para o usuário visualizar seu próprio currículo)

Etapa 5: Estilização e UX

- Utilize componentes do Angular Material (cards, formulários, botões) para criar uma interface amigável para o cadastro, edição e visualização dos currículos.
- Implemente MatSnackBar para exibir mensagens de sucesso ou erro após as operações.

7. Competências Avaliadas

Esta situação de aprendizagem permitirá avaliar as seguintes competências:

1. Desenvolvimento Frontend com Angular:
 - Uso do Angular CLI para criação de projetos, módulos e componentes.
 - Compreensão da arquitetura de componentes e comunicação entre eles.
 - Aplicação de módulos e serviços.
2. Manipulação de Dados e Requisições HTTP:

- Consumo de APIs RESTful usando HttpClientModule.
 - Realização de operações CRUD (GET, POST, PUT, DELETE) com dados.
 - Tratamento de dados assíncronos com Observables (rxjs).
 - 3. Gerenciamento de Estado e Formulários:
 - Criação e validação de formulários reativos.
 - Exibição de feedback ao usuário sobre o estado das operações.
 - 4. Roteamento e Navegação:
 - Configuração e uso do Angular Router para navegação entre telas.
 - Passagem de parâmetros via rota.
 - 5. Organização e Boas Práticas de Código:
 - Estruturação lógica do projeto (pastas core, shared, models, módulos por funcionalidade).
 - Criação de interfaces para modelos de dados.
 - Reuso de componentes e serviços.
 - 6. Resolução de Problemas:
 - Identificação e correção de erros comuns no desenvolvimento frontend.
 - Adaptação a um backend simulado (json-server).
-

8. Critérios de Avaliação

O projeto será avaliado com base nos seguintes critérios:

1. Funcionalidade:
 - Cadastro/Edição de Currículo: Formulário funcional, com validações adequadas, que permite criar e editar informações do currículo (formação, experiência, habilidades, LinkedIn).
 - Visualização de Currículo: O usuário consegue visualizar seu próprio currículo de forma clara e organizada.
 - Listagem Simulada de Currículos para Empresas : A aplicação consegue, de forma simulada, buscar e exibir os currículos dos usuários
2. Qualidade do Código:
 - Organização e Estrutura: Código bem organizado em módulos e componentes, seguindo a estrutura proposta (models, views e funcionalidades específicas).
 - Legibilidade e Manutenibilidade: Código claro, com nomes de variáveis e funções significativos, e comentários onde necessário.

3. Uso de Tecnologias:

- Angular: Aplicação correta de conceitos fundamentais do Angular (componentes, serviços, módulos, roteamento, formulários reativos, lifecycle hooks).
- json-server e HTTP: Interação eficiente e correta com o json-server, utilizando HttpClientModule para todas as operações.

4. Experiência do Usuário (UX) e Design:

- Navegação: Rotas claras e intuitivas para acesso às funcionalidades de currículos.
- Feedback ao Usuário: Mensagens de sucesso, erro ou carregamento visíveis e compreensíveis (ex: SnackBar, alerts).

Links Para Entregas

Autenticação de Usuário Com Angular

Explicação Teórica: Arquitetura de Autenticação em Aplicações Web Modernas

O projeto que desenvolvemos, embora simples, implementa um padrão de arquitetura fundamental para praticamente qualquer aplicação web que lida com dados de usuário: o fluxo de **autenticação e autorização**. Vamos desconstruir os conceitos teóricos por trás dele.

1. O Pilar da Segurança: Autenticação vs. Autorização

É crucial entender a diferença entre estes dois termos, que muitas vezes são confundidos.

- **Autenticação: "Quem é você?"**

É o processo de **verificar a identidade** de um usuário. Quando um usuário insere um e-mail e uma senha, a aplicação confere se essas credenciais correspondem a um registro válido em seu banco de dados (no nosso caso, o db.json). Se corresponderem, o usuário é considerado "autenticado". O resultado da autenticação é binário: ou você é quem diz ser, ou não é.

- **Analogia:** Mostrar sua carteira de identidade para provar seu nome e idade.

- **Autorização: "O que você tem permissão para fazer?"**

Este processo ocorre **após** uma autenticação bem-sucedida. Ele define quais ações, recursos ou páginas um usuário autenticado pode acessar. Em sistemas mais complexos, a autorização pode ser granular, baseada em "papéis" ou "funções" (como usuário, moderador, administrador).

- **Analogia:** Sua identidade foi verificada (autenticação). Agora, seu crachá (autorização) permite que você entre no prédio principal, mas apenas um crachá de "Nível 5" permite o acesso à sala do servidor.

No nosso projeto, a **autenticação** é feita na tela de login, e a **autorização** é o que nos permite (ou não) acessar a rota /interna.

2. O Guardião das Rotas: O Papel do Middleware (Angular Route Guards)

Como uma aplicação garante que um usuário não autenticado não acesse uma página protegida simplesmente digitando a URL no navegador? A resposta é o **Middleware**.

- **O que é Middleware?**

Middleware é um software ou um trecho de código que atua como uma ponte entre duas partes de uma aplicação. Ele se posiciona no meio de um fluxo de requisição/resposta para executar tarefas específicas. Pense nele como um **ponto de checagem de segurança** ou um pedágio em uma rodovia. Antes que a sua solicitação (navegar para uma nova página) chegue ao seu destino final (o componente daquela página), ela deve passar pelo middleware.

- **Angular Route Guards como Middleware**

No ecossistema Angular, os **Route Guards** são a implementação específica do conceito de middleware para o módulo de roteamento. Eles são classes que executam uma lógica e retornam true (permitindo a navegação) ou false (bloqueando a navegação).

No nosso projeto, o AuthGuard implementa a interface CanActivate. O seu funcionamento é o seguinte:

1. **Interceptação:** Quando o usuário tenta navegar para a rota /interna, o Roteador do Angular não carrega o InternaComponent imediatamente. Primeiro, ele aciona o AuthGuard.
2. **Verificação:** O método canActivate dentro do AuthGuard é executado. Ele chama nosso AuthService para fazer a pergunta crucial: estaAutenticado()?.
3. **Decisão:**
 - Se o serviço retornar true, o AuthGuard também retorna true. O Roteador recebe a luz verde e permite que a navegação para o InternaComponent continue.
 - Se o serviço retornar false, o AuthGuard executa uma ação secundária (redirecionar para /login) e então retorna false. O Roteador recebe a luz vermelha, cancela a navegação original para /interna e executa a nova navegação para a página de login.

Essa abordagem é extremamente poderosa e limpa, pois desacopla a lógica de proteção da lógica do próprio componente. O InternaComponent não precisa saber se o usuário está logado ou não; ele simplesmente "confia" que, se foi renderizado, é porque o AuthGuard já fez seu trabalho.

Exemplos de Aplicação e Importância

A combinação de autenticação e middleware de rota não é um luxo, mas uma necessidade para a vasta maioria das aplicações web.

Exemplo 1: E-commerce (ex: Amazon)

- **Páginas Públicas:** Home, busca de produtos, páginas de detalhes de produtos. Não exigem login.
- **Middleware em Ação:** Quando você clica em "Minha Conta", "Meus Pedidos" ou "Finalizar Compra".
- **Importância:**
 - **Autenticação:** Garante que você está acessando seus próprios dados de pedido, endereço e pagamento. Sem isso, qualquer um poderia ver e alterar suas informações.

- **Autorização:** Permite que você veja os *seus* pedidos, mas não os de outros usuários. Permite que você edite os *seus* endereços.
- **Proteção de Dados:** Protege informações financeiras e pessoais, sendo um requisito legal em muitas jurisdições (como a LGPD no Brasil).

Exemplo 2: Plataforma de Streaming (ex: Netflix)

- **Páginas Públicas:** A página inicial de marketing que descreve os planos.
- **Middleware em Ação:** Praticamente toda a aplicação após a tela inicial. A tentativa de acessar a tela de seleção de perfis ou o catálogo de filmes aciona uma guarda que verifica se há uma sessão de login ativa.
- **Importância:**
 - **Controle de Acesso:** Garante que apenas assinantes pagantes tenham acesso ao conteúdo.
 - **Personalização:** A autenticação permite que a plataforma saiba quem você é para carregar sua lista de "Continuar Assistindo", suas preferências e suas recomendações personalizadas. Sem autenticação, essa experiência seria impossível.

Exemplo 3: Sistema Corporativo / Dashboard (ex: Salesforce, um CRM)

- **Páginas Públicas:** Geralmente, apenas a página de login.
- **Middleware em Ação:** Em todas as rotas internas. Aqui, o middleware pode ser ainda mais complexo.
- **Importância:**
 - **Segurança Corporativa:** Impede o acesso não autorizado a dados confidenciais de clientes e vendas.
 - **Autorização Baseada em Papéis (RBAC):** Um AuthGuard mais avançado pode verificar não apenas se o usuário está logado, mas também qual é o seu "papel" (vendedor, gerente, diretor).
 - Um **vendedor** pode ver e editar apenas os seus próprios clientes.
 - Um **gerente**, ao acessar a mesma rota /dashboard, pode ver os dados consolidados de toda a sua equipe.
 - Um **diretor** pode ter acesso a ferramentas de administração que são invisíveis para os outros papéis.

O middleware é o ponto de controle perfeito para implementar essa lógica de autorização complexa.

Conclusão

A arquitetura de **Autenticação + Middleware (Route Guards)** é um pilar do desenvolvimento de software seguro e funcional. Ela permite criar aplicações que não apenas protegem os dados, mas também oferecem experiências ricas e personalizadas, separando de forma clara e eficiente as responsabilidades: os componentes cuidam da interface, os serviços cuidam da lógica de negócio e os guardiões (middleware) cuidam da segurança e do controle de acesso às rotas.

Fazendo um Projeto com Autenticação e Autorização

Estrutura do Projeto Final

- **Front-end (Angular):** Gerencia a interface do usuário, formulários, rotas e a comunicação com a API.
- **Back-end Simulado** (Fornece uma API RESTful para registrar e buscar usuários, lendo e escrevendo em um arquivo `db.json`.

https://docs.google.com/document/d/1dfvSQd_HZ2qomL_Vq6toz5K9WHFiSDzu49q-yWUpzzM/edit?tab=t.qanawoxy3jhi

Anexo 1 - TypeScript

Introdução ao TypeScript e suas diferenças em relação ao JavaScript

Objetivo

- Compreender o que é TypeScript.
 - Entender suas vantagens sobre o JavaScript.
 - Identificar suas principais diferenças.
 - Aplicar conceitos básicos com exemplos práticos.
-

O que é o TypeScript?

- TypeScript é um **superset do JavaScript** criado pela Microsoft.
- Ele adiciona **tipagem estática opcional** ao JavaScript.
- Compila para JavaScript padrão, rodando em qualquer navegador ou ambiente Node.js.

"Todo código TypeScript é JavaScript, mas nem todo JavaScript é TypeScript."

Diferenças entre JavaScript e TypeScript

Característica	JavaScript	TypeScript
Tipagem	Dinâmica	Estática
Tempo de Erro	Em tempo de execução	Em tempo de compilação
Autocompletar	Limitado	Rico e preciso
IDE/Editor	Pode não mostrar erros	IntelliSense e verificação de tipos

Orientação a Objetos	Limitada	Suporte completo (interfaces, generics, etc)
----------------------	----------	--

Exemplos Comparativos

Exemplo 1 - Declaração de Variáveis

JavaScript:

```
let nome = "Maria";  
nome = 123; // Funciona, mas pode gerar erro em tempo de execução
```

TypeScript:

```
let nome: string = "Maria";  
nome = 123; // Erro: tipo 'number' não é atribuível a 'string'
```

Exemplo 2 - Funções com Tipos

JavaScript:

```
function soma(a, b) {  
  return a + b;  
}  
  
console.log(soma("2", 3)); // Resultado: "23"
```

TypeScript:

```
function soma(a: number, b: number): number {  
  return a + b;  
}  
  
// console.log(soma("2", 3)); // Erro de compilação
```

Exemplo 3 - Interfaces e Objetos

TypeScript:

```
interface Pessoa {  
  nome: string;  
  idade: number;  
}  
  
function exibirPessoa(p: Pessoa) {  
  console.log(`${p.nome} tem ${p.idade} anos.`);  
}  
  
const joao = { nome: "João", idade: 25 };  
exibirPessoa(joao);
```

Vantagens do TypeScript

- **Detecção de erros antes de executar**
- **Melhor suporte em IDEs**
- **Mais organizado e escalável**
- Ideal para **projetos grandes ou em equipe**

Conclusão

- TypeScript não substitui o JavaScript, ele o **complementa**.
 - Você ganha **mais segurança**, organização e clareza no código.
 - É uma excelente escolha para aplicações de médio e grande porte.
-

Passos para Preparar o Ambiente de Desenvolvimento TypeScript

Instalar o Node.js

```
node -v  
npm -v
```

Instalar o TypeScript Globalmente

```
npm install -g typescript
```

Verifique se instalou corretamente:

```
tsc -v
```

Criar o Projeto TypeScript

Crie uma nova pasta para o seu projeto e entre nela:

```
mkdir meu-projeto-ts  
cd meu-projeto-ts
```

Inicialize o projeto com npm:

```
npm init -y
```

Instalar o TypeScript localmente (projeto)

Apesar de estar instalado globalmente, é boa prática instalar localmente no projeto:

```
npm install typescript --save-dev
```

Gerar o arquivo de configuração tsconfig.json

```
npx tsc --init
```

Esse comando cria um arquivo tsconfig.json com várias opções de compilação.

Dica: Edite o arquivo para ativar boas práticas (exemplo abaixo).

Configurar tsconfig.json (opcional, recomendado)

Aqui está uma configuração inicial sugerida:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true
  },
  "include": ["src"]
}
```

Criar estrutura de pastas

```
mkdir src
```

Crie um arquivo inicial, por exemplo:

```
type nul > src/index.ts
```

Compilar o projeto

Para compilar os arquivos .ts para .js:

```
npx tsc
```

Ou, para compilar e observar automaticamente:

```
npx tsc --watch
```

O código compilado será gerado na pasta dist/ (como configurado).

Executar o JavaScript gerado

```
node dist/index.js
```

(Opcional) Adicionar scripts no package.json

```
"scripts": {  
  "start": "node dist/index.js",  
  "build": "tsc",  
  "dev": "tsc --watch"  
}
```

Agora você pode rodar:

```
npm run dev
```

Atividade Prática

Exercício 1: Crie um arquivo TypeScript que:

- Declare uma interface Produto com nome, preco e categoria.

- Implemente uma função que recebe um produto e exibe suas informações.

Exercício 2: Crie uma função `calcularDesconto(preco: number, percentual: number)` e use-a com produtos diferentes.

Exercícios - Para aprender +

https://docs.google.com/document/d/1nHmCV7lolYfNzyDwcYswdPsHc92TgxEr_nisAt_hulfo/edit?tab=t.rdp3xiyi590s

Anexo 2 - Json-Server

O que é o JSON-Server?

O json-server é uma biblioteca Node.js que transforma um arquivo .json em uma **API REST completa**. Ele é usado para simular um backend em desenvolvimento front-end, especialmente útil para testes, protótipos ou até integração contínua de software.

Por que usar o json-server?

✓ Vantagens:

- Simples e rápido de configurar.
 - Gera automaticamente rotas REST (GET, POST, PUT, DELETE, PATCH).
 - Suporte a filtros e paginação.
 - Pode simular atrasos de rede, autenticação etc.
 - Ótimo para testes com **Angular, React, Vue, etc.**
-

Como instalar e configurar

1. Instalar globalmente ou no projeto:

```
npm install -g json-server  
# ou localmente:  
npm install json-server --save-dev
```

2. Criar um arquivo db.json com sua base de dados simulada:

```
{
  "usuarios": [
    { "id": 1, "nome": "Maria", "email": "maria@mail.com" },
    { "id": 2, "nome": "João", "email": "joao@mail.com" }
  ],
  "produtos": [
    { "id": 1, "nome": "Notebook", "preco": 3000 },
    { "id": 2, "nome": "Mouse", "preco": 150 }
  ]
}
```

3. Rodar o servidor:

```
json-server --watch db.json
```

Ele ficará disponível por padrão em <http://localhost:3000>.

Rotas REST automáticas

Método	Rota	Ação
GET	/usuarios	Lista todos os usuários
GET	/usuarios/1	Detalhe do usuário com ID 1
POST	/usuarios	Adiciona novo usuário
PUT	/usuarios/1	Atualiza o usuário com ID 1
PATCH	/usuarios/1	Atualiza parcialmente o usuário

DELETE	/usuarios/1	Remove o usuário com ID 1
--------	-------------	---------------------------

Testando com Thunder Client (ou Postman)

O **Thunder Client** é uma extensão do VS Code que facilita testes de APIs.

Exemplos de uso:

Buscar todos os produtos (GET)

GET http://localhost:3000/produtos

Criar um novo produto (POST)

POST http://localhost:3000/produtos

Body (JSON):

```
{  
  "nome": "Teclado",  
  "preco": 200  
}
```

Atualizar (PUT/PATCH)

PUT http://localhost:3000/produtos/2

Body:

```
{  
  "nome": "Mouse Gamer",  
  "preco": 250  
}
```

```
}
```

Deletar

DELETE http://localhost:3000/produtos/2

Usos na modelagem de software

Em modelagem ágil ou testes de frontend, usar o json-server é excelente porque:

- Permite criar **protótipos rápidos**.
 - Ajuda a simular dados reais sem backend pronto.
 - Facilita o trabalho em **equipe** (front x back separados).
 - Serve como base para **mock de serviços** em testes automatizados.
 - Pode ser versionado com Git, como parte do projeto.
-

Exemplo com Angular

Você pode consumir os dados com Angular usando o HttpClient:

```
@Injectable({ providedIn: 'root' })  
export class ProdutoService {  
  private api = 'http://localhost:3000/produtos';
```

```
constructor(private http: HttpClient) {}

getProdutos(): Observable<Produto[]> {
  return this.http.get<Produto[]>(this.api);
}

adicionarProduto(produto: Produto): Observable<Produto> {
  return this.http.post<Produto>(this.api, produto);
}

atualizarProduto(produto: Produto): Observable<Produto> {
  return this.http.put<Produto>(`${this.api}/${produto.id}`, produto);
}

deletarProduto(id: number): Observable<void> {
  return this.http.delete<void>(`${this.api}/${id}`);
}
}
```

Conclusão

O json-server é uma **ferramenta leve, versátil e essencial** para quem desenvolve com Angular ou qualquer framework frontend. Ele reduz a dependência imediata do backend, acelera testes, protótipos e promove melhor arquitetura e produtividade na equipe.

SA: Plataforma RH

Situação de Aprendizagem: "Plataforma RH: Cadastro de Vagas e Currículos com Acesso Autenticado"

Objetivo Geral

Desenvolver uma aplicação Angular com back-end simulado (json-server) que permita a gestão de vagas e currículos, utilizando autenticação (login/registro) e autorização por tipo de usuário (Admin e Usuário Comum). O projeto será orientado por etapas de levantamento de requisitos, prototipagem, diagramação e codificação.

Objetivos Específicos

- Compreender a diferença entre **autenticação e autorização**.
 - Implementar **guardas de rota** com base em perfis de usuário (admin vs comum).
 - Utilizar **formulários reativos** com validação no Angular.
 - Simular um back-end usando json-server.
 - Criar **componentes reutilizáveis** e manter estrutura de código organizada.
 - Aplicar **SCSS customizado** para layout responsivo e moderno.
 - Desenvolver uma aplicação funcional a partir de um **briefing realista**.
-



Etapa 1: Enunciado da Situação / Desafio

A empresa fictícia **RH Connect** deseja um protótipo funcional de seu novo sistema interno de gestão de recrutamento. O sistema deve permitir:

- **Cadastro e login de usuários comuns**, que poderão preencher e enviar seus **currículos**.
- **Login de administradores**, com permissão para **criar, editar e excluir vagas**.
- Listagem pública das vagas.
- **Restrições de acesso** baseadas no tipo de usuário autenticado.

O produto final será um SPA (Single Page Application) desenvolvida com Angular.

Etapa 2: Levantamento do Escopo

Funcionalidades Requeridas

Usuário Comum

- Cadastro de conta
- Login
- Cadastro de currículo
- Visualização de vagas

Administrador

- Login
- Cadastro, edição e exclusão de vagas
- Visualização de currículos recebidos

Estrutura dos Dados (db . json)

{

```
"usuarios": [  
  {  
    "id": 1,  
    "email": "admin@rh.com",  
    "senha": "admin123",  
    "tipo": "admin"  
  }  
],  
"curriculos": [],  
"vagas": []  
}
```

Etapa 3: Briefing de Apresentação

Cliente: RH Connect

Responsável pelo projeto: Equipe de Desenvolvimento Angular

Prazo estimado: 2 semanas

Objetivo: Construir uma aplicação de RH funcional que simule um sistema real com login, autorização por perfil e funcionalidades específicas para cada tipo de usuário.

Público-Alvo: Candidatos (usuários comuns) e administradores da empresa.

Tons da marca: Azul-escuro (#002D62), Cinza-claro (#f4f4f4), Branco

Dispositivos-alvo: Desktop e dispositivos móveis (design responsivo).

Etapa 4: Protótipos e Diagramas (Obrigatórios)

Protótipos (UI/UX)

- Tela de login
- Tela de registro
- Área administrativa (vagas)
- Área do candidato (currículo)

- Página pública com listagem de vagas

Ferramentas sugeridas: Figma,

Diagrama de Fluxo

- Fluxo de autenticação
- Fluxo de navegação por perfil

Diagrama de Uso

Diagrama de Classe - > Relacionamento Entre Entidades

Etapa 5: Desenvolvimento Angular

Tecnologias

- Angular 16+
 - SCSS customizado
 - JSON Server
 - Reactive Forms
 - LocalStorage para sessão
 - Guardas de rota
-

Estrutura Sugerida

```
pgsql  
CopiarEditar  
src/
```

```
|
|
|— app/
|   |— componentes/
|   |   |— login/
|   |   |— registro/
|   |   |— vagas/
|   |   |— curriculo/
|   |   |— admin-vagas/
|   |   └─ navbar/
|   |— servicos/
|   |   └─ auth.service.ts
|   |   └─ vagas.service.ts
|   |   └─ curriculo.service.ts
|   |— guardas/
|   |   └─ auth.guard.ts
|   └─ modelos/
|       └─ usuario.model.ts
```

Componentes e Funcionalidades

Componente	Função
login	Login com validação e redirecionamento por perfil
registro	Cadastro de usuários (define tipo "comum")
navbar	Navegação condicional (comum/admin/deslogado)
vagas	Página pública de vagas
curriculo	Formulário de envio de currículo (somente comum)
admin-vagas	CRUD de vagas (somente admin)

Segurança

- Guarda AuthGuard verifica se usuário está logado.

- Guarda AdminGuard verifica tipo do usuário.
 - Navegação e rotas condicionadas ao perfil do usuário.
-

Regras de Acesso (Exemplo de Guarda Avançada)

```
canActivate(route: ActivatedRouteSnapshot): boolean {  
  const tipo = this.authService.usuarioAtual()?.tipo;  
  const autorizado = route.data['autorizado'];  
  return tipo === autorizado;  
}
```

Etapa 6: Avaliação

Critério

Protótipo e diagramas entregues

Estrutura e organização do projeto Angular

Funcionalidade de autenticação e autorização

Implementação correta de formulários e CRUDs

Estilização com SCSS personalizado e responsivo

Entregáveis

1. Documentação com Levantamento de Requisitos (Sem Análise de Risco)
 - Escopo, Objetivos Smart, Levantamento de Requisitos (Recursos humanos e Ferramentas de Desenvolvimento), Cronograma
2. Protótipos (Figma ou imagem)
3. Diagramas

4. Código-fonte do projeto Angular (repositório Git)
5. Gravação de tela com apresentação demonstrando a funcionalidade

Dicas Técnicas

- Utilize `localStorage` para salvar dados da sessão.
 - Implemente `CanActivate` e `CanLoad` se quiser um controle mais refinado.
 - Use `formBuilder.group()` para validação de formulários.
 - SCSS com `:root`, `mixins` e `variables` deixará seu layout mais profissional.
-

Castello Turma A

Grupo 1 - Anthony, Giovanna G, Giovanna M, Julia(PO)
Grupo 2 - Iasmin(PO), Isabela R, Gleice, Mioto
Grupo 3 - Gabriel D, Gabriel E, Isabela F(PO), Manuela
Grupo 4 - Daniel(PO), Gullit, Esther, Laiza

Castello Turma B - Entrega 20/08

Grupo 1 - Matheus B, Matheus C(PO), Pedro, Rayssa
Grupo 2 - Anick, Luiza F, Marco Antonio(PO), Mirella
Grupo 3 - Luiza S, Maria Clara, Paula(PO), Thiago
Grupo 4 - Lívia, Pietro(PO), Renan, Vitor

SESI Turma A

Grupo 1 - Guilherme(PO), Julya, Derick, Enzo D
Grupo 2 - Aron, Clara, Beatriz(PO), João C
Grupo 3 - Enzo P, Kauan S, Kauã R, Isabela(PO)
Grupo 4 - Adriely(PO), Julia, Endrew, Eduardo

SESI Turma B

Grupo 1 - Maria , Luiza, Leonardo(PO), Vitor R
Grupo 2 - Lucas, Pedro G, Pedro P, Pedro C(PO)
Grupo 3 - Vitor G, Nathaly, Matheus(PO), Thauany
Grupo 4 - Miguel(PO), Lucas P, Rafael

Correção SA: PlataformaRH

Passo a Passo da Codificação do Projeto "Plataforma RH"

Pré-requisito: Configuração do Ambiente

Antes de começar, certifique-se de ter o Node.js e o Angular CLI instalados.

1. Criar o Projeto Angular:

2. codeBash

ng new plataforma-rh --style=scss

3. cd plataforma-rh

4. Isso cria um novo projeto Angular chamado plataforma-rh e já o configura para usar SCSS.

5. Configurar o JSON Server (Backend Simulado):

○ Instale o json-server:

6. Criar o BackEnd Simulado : (Arquivo db.json)

```
{
  "usuarios": [
    {
      "id": 1,
      "nome": "Administrador",
      "email": "admin@rh.com",
      "senha": "admin123",
      "tipo": "admin"
    }
  ],
  "curriculos": [],
  "vagas": [
    {
      "id": 1,
      "titulo": "Desenvolvedor Angular Pleno",
      "descricao": "Procuramos um desenvolvedor com 3+ anos de experiência.",
      "salario": 7500,
      "localizacao": "Remoto"
    }
  ]
}
```

Etapa 1: Estrutura Fundamental (Modelos e Serviços)

Vamos criar a base da nossa aplicação.

1. Definir os Modelos:

Crie os arquivos para os modelos de dados.

src/app/models/usuario.model.ts:

```
export class Usuario {  
  id: number;  
  nome: string;  
  email: string;  
  senha?: string; // Senha é opcional, pois não queremos que ela  
  trafegue de volta do backend  
  tipo: 'admin' | 'comum';  
}
```

src/app/models/[vaga.model.ts](#)

```
export class Vaga {  
  id: number;  
  titulo: string;  
  descricao: string;  
  salario: number;  
  localizacao: string;  
}
```

src/app/models/[curriculo.model.ts](#):

```
export class Curriculo{  
  id: number;  
  nome: string;  
  email: string;  
  telefone: string;  
  experiencia: string;
```

```
    formacao: string;
    habilidades: string[];
  }
```

2. Gerar os Serviços:

Usaremos o Angular CLI para criar os serviços que farão a comunicação com nosso backend.

```
ng generate service servicos/auth
```

```
ng generate service servicos/vagas
```

```
ng generate service servicos/curriculo
```

3. Adicionar o HttpClientModule ao src/app/[app.module.ts](#):

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    HttpClientModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

4. Implementar o AuthService

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { BehaviorSubject, Observable, tap } from 'rxjs';
import { Usuario } from '../models/usuario.model';
import { Router } from '@angular/router';

@Injectable({
  providedIn: 'root',
})
export class AuthService {
  // URL da API
  private apiUrl = 'http://localhost:3000/usuarios';
  // Subject para armazenar o usuário atual
  private usuarioAtualSubject = new BehaviorSubject<Usuario | null>(null);
  // Observable para expor o usuário atual
  public usuarioAtual$ = this.usuarioAtualSubject.asObservable();

  // Construtor
  constructor(private http: HttpClient, private router: Router) {
    const usuario = localStorage.getItem('usuario_logado');
    if (usuario) {
      this.usuarioAtualSubject.next(JSON.parse(usuario));
    }
  }

  // Getter para o usuário atual
  public get usuarioAtualValue(): Usuario | null {
    return this.usuarioAtualSubject.value;
  }

  // Método para fazer login
  login(email: string, senha: string): Observable<Usuario[]> {
    return this.http
      .get<Usuario[]>(`${this.apiUrl}?email=${email}&senha=${senha}`)
      .pipe(
        tap((usuarios) => {
          if (usuarios.length > 0) {
            const usuario = usuarios[0];

```

```

        localStorage.setItem('usuario_logado',
JSON.stringify(usuario));
        this.usuarioAtualSubject.next(usuario);
    }
    })
    );
}

// Método para registrar um novo usuário
registrar(usuario: Omit<Usuario, 'id'>): Observable<Usuario> {
    return this.http.post<Usuario>(this.apiUrl, usuario);
}

// Método para fazer logout
logout(): void {
    localStorage.removeItem('usuario_logado');
    this.usuarioAtualSubject.next(null);
    this.router.navigate(['/login']);
}

// Método para verificar se o usuário está autenticado
isAuthenticated(): boolean {
    return !!this.usuarioAtualValue;
}

// Método para verificar se o usuário é admin
isAdmin(): boolean {
    return this.usuarioAtualValue?.tipo === 'admin';
}
}

```

5. Criar os componentes para Login e Registro

ng g c pages/login

ng g c pages/registro

src/app/pages/[login.component.ts](#):

```
import { Component, OnInit } from '@angular/core';
```

```
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from 'src/app/services/auth.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.scss'],
})
export class LoginComponent implements OnInit {
  loginForm!: FormGroup;
  erroLogin = false;

  constructor(
    private fb: FormBuilder,
    private authService: AuthService,
    private router: Router
  ) {}

  ngOnInit(): void {
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      senha: ['', [Validators.required]],
    });
  }

  onSubmit(): void {
    if (this.loginForm.invalid) {
      return;
    }
    this.erroLogin = false;
    const { email, senha } = this.loginForm.value;

    this.authService.login(email, senha).subscribe((usuarios) => {
      if (usuarios.length > 0) {
        const usuario = usuarios[0];
        if (usuario.tipo === 'admin') {
          this.router.navigate(['/admin/vagas']);
        } else {
          this.router.navigate(['/curriculo']);
        }
      }
    });
  }
}
```

```

    }
  } else {
    this.erroLogin = true;
  }
});
}
}

```

6. Criar os Guard para Proteger as Rotas

ng g guard guards/auth --functional=false

ng g guard guards/admin --functional=false

src/app/guards/[auth.guard.ts](#):

```

import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Injectable({
  providedIn: 'root',
})
export class AuthGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.authService.isAuthenticated()) {
      return true;
    } else {
      this.router.navigate(['/login']);
      return false;
    }
  }
}

```

src/app/guards/[admin.guard.ts](#):

```

import { Injectable } from '@angular/core';

```

```
import { CanActivate, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Injectable({
  providedIn: 'root',
})
export class AdminGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.authService.isAuthenticated() && this.authService.isAdmin()) {
      return true;
    } else {
      // Poderia redirecionar para uma página de "acesso negado"
      this.router.navigate(['/vagas']);
      return false;
    }
  }
}
```

Etapa 2 - Implementar os sistema Vagas e Currículo Já Criado

1. Implementando as Funcionalidades de Admin (CRUD de Vagas)
2. Implementar as Funcionalidade de Cliente(CRUD de Currículo)

https://drive.google.com/file/d/126bIKTHIFodJU5YaJE9crRED3jCsd4v3/view?usp=drive_link - link para o arquivo com o passo-a-passo para a atividade

Etapa 4 - Criar as rotas de Navegação

1. Implementar as rotas de Navegação no Arquivo app.routing.ts:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { LoginComponent } from '../pages/login/login.component';
import { RegistroComponent } from '../pages/registro/registro.component';
import { VagasComponent } from '../pages/vagas/vagas.component';
```

```

import { CurriculoComponent } from
'./pages/curriculo/curriculo.component';
import { AdminVagasComponent } from
'./pages/admin-vagas/admin-vagas.component';
import { AuthGuard } from './guards/auth.guard';
import { AdminGuard } from './guards/admin.guard';

const routes: Routes = [
  { path: 'login', component: LoginComponent },
  { path: 'registro', component: RegistroComponent },
  { path: 'vagas', component: VagasComponent },
  {
    path: 'curriculo',
    component: CurriculoComponent,
    canActivate: [AuthGuard], // Protegida por login
  },
  {
    path: 'admin/vagas',
    component: AdminVagasComponent,
    canActivate: [AuthGuard, AdminGuard], // Protegida por login e perfil
de admin
  },
  { path: '', redirectTo: '/vagas', pathMatch: 'full' },
  { path: '**', redirectTo: '/vagas' }, // Rota coringa
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule],
})
export class AppRoutingModule {}

```

Etapa 3 - Navegação Condicional e Estilização

1. Gerar o Componente NavBar em Template
ng g c pages/templates/navbar

2. Implementar a Lógica de Navegação do NavBar

src/app/pages/templates/[navbar.component.ts](#)

```
import { Component } from '@angular/core';
import { AuthService } from 'src/app/services/auth.service';

@Component({
  selector: 'app-navbar',
  templateUrl: './navbar.component.html',
  styleUrls: ['./navbar.component.scss'],
})
export class NavbarComponent {
  // Tornamos o serviço público para que o template possa acessá-lo
  // diretamente
  constructor(public authService: AuthService) {}

  onLogout(): void {
    this.authService.logout();
  }
}
```

3. Criar o Html utilizando DataBinding para manipular a navegação conforme o usuário conectado

src/app/pages/templates/navbar.component.html

```
<nav>
  <ul>
    <li><a routerLink="/vagas">Ver Vagas</a></li>

    <!-- Links para usuário comum logado -->
    <ng-container *ngIf="authService.isAuthenticated() &&
!authService.isAdmin()">
      <li><a routerLink="/curriculo">Meu Currículo</a></li>
    </ng-container>

    <!-- Links para Admin -->
    <ng-container *ngIf="authService.isAdmin()">
      <li><a routerLink="/admin/vagas">Gerenciar Vagas</a></li>
    </ng-container>
  </ul>
</nav>
```

```

<!-- Links para Deslogado -->
<ng-container *ngIf="!authService.isAuthenticated()">
  <li><a routerLink="/login">Login</a></li>
  <li><a routerLink="/registro">Registrar</a></li>
</ng-container>

<!-- Botão de Logout para Logado -->
<li *ngIf="authService.isAuthenticated()">
  <button (click)="onLogout()">Sair</button>
</li>
</ul>
</nav>

```

4. Criar a Estilização em SCSS:

```

:root {
  --cor-primaria: #002D62;
  --cor-fundo: #f4f4f4;
  --cor-texto: #333;
}

```